

Dart Tutorial 4: Conditional Statements

This chapter explains how to control the flow of a Dart program using conditional statements. After completing this tutorial, students will understand how to make decisions in their code using `if`, `else if`, `else`, and `switch` statements.

1. Introduction to Conditional Statements

A **conditional statement** allows a program to make decisions and execute different blocks of code based on whether a condition is true or false.

Without conditional statements, a program would always run the same way, with no ability to react to different situations.

2. Why We Use Conditional Statements

In real world apps, we constantly need to make decisions, such as checking login status or validating user input.

Conditional statements allow us to:

- Execute code only when a certain condition is met.
 - Choose between multiple possible outcomes.
 - Validate data before processing it.
 - Control the flow of an entire application.
-

3. The `if` Statement

The simplest conditional statement is `if`, which runs a block of code only when its condition is true.

```
int age = 20;

if (age >= 18) {
  print('You are an adult');
}
```

Output:

```
You are an adult
```

4. The if-else Statement

The `else` block runs when the `if` condition is false.

```
int age = 15;

if (age >= 18) {
  print('You are an adult');
} else {
  print('You are a minor');
}
```

Output:

```
You are a minor
```

5. The if-else if-else Statement

When there are multiple conditions to check, we use `else if`.

```
int marks = 75;

if (marks >= 90) {
  print('Grade A');
} else if (marks >= 70) {
  print('Grade B');
} else {
  print('Grade C');
}
```

Output:

```
Grade B
```

6. Nested if Statements

An `if` statement can be placed inside another `if` statement to check multiple related conditions.

```
int age = 25;
bool hasLicense = true;

if (age >= 18) {
  if (hasLicense) {
    print('You can drive');
  } else {
    print('You need a license');
  }
}
```

Output:

```
You can drive
```

7. The switch Statement

The **switch** statement is used when we need to compare one value against multiple fixed options.

```
String day = 'Monday';

switch (day) {
  case 'Monday':
    print('Start of the week');
    break;
  case 'Friday':
    print('Almost weekend');
    break;
  default:
    print('Just another day');
}
```

Output:

```
Start of the week
```

8. if-else vs switch

Feature	if-else	switch
Best For	Range based conditions	Fixed value comparisons
Readability	Less clear with many conditions	More clear for many fixed values
Performance	Slightly slower with many checks	Slightly faster for many cases
Default Case	Uses else	Uses default

9. Ternary Operator (Short Conditional)

Dart provides a shorthand way to write simple if-else logic.

```
int age = 20;
String result = age >= 18 ? 'Adult' : 'Minor';

print(result);
```

Output:

```
Adult
```

10. Conclusion of Tutorial 4

In this tutorial, students have learned:

1. What conditional statements are and why they are important.
2. How to use if, if-else, and else if for decision making.
3. How to use nested if statements for multiple related conditions.
4. How to use switch statements for fixed value comparisons.
5. How to write quick conditions using the ternary operator.

In the next tutorial, we will learn about Loops in Dart.

This completes Dart Tutorial 4.